NEXT ENGINEER · _SHEERAT_

Design and Development of Applications

BCS073

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT | **Semester:** 7 | **Credits:** 3 | **L-T-P:** 3-0-0

Scheme: AKTU B.Tech 4th year, Departmental Elective-IV, 2025-26 onwards

UNIT 1 Introduction to Mobile Applications

UNIT 2 Basic Design and Quality Constraints

UNIT 3 Advanced Design

UNIT 4 Technology I: Android

UNIT 5 Technology II: iOS

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: architectures, life cycles and comparison tables.
3. **Examples and comparisons**: the design scenarios and small calculations that appear in the paper (app size and storage, frame budget, battery drain, availability, layout choices).
4. **Code / key points to remember**: Android (Kotlin and Java), SQLite, Objective-C and Swift snippets you can reproduce.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1 (mobile apps, embedded systems, market and business drivers, publishing and delivery, requirements gathering and validation) and Unit 2 (embedded design, embedded OS, constraints, architecting mobile apps, UI, touch events and gestures, quality attributes).
- **Afternoon (3 hours):** Unit 3 (multimedia, web access, GPS, social networking, cloud-hosted apps, design patterns) and Unit 4 (Android architecture, activities, views, intents, SQLite, packaging, server interaction, maps, GPS, Wi-Fi, social media).
- **Evening (2 hours):** Unit 5 (iOS, Objective-C, UI and touch frameworks, Core Data, Core Location, Map Kit, calendar and contacts, App Store, Swift). Then the "Last-hour checklist".
- **In the exam:** draw the Android architecture stack and the Activity life cycle every time; they are asked almost every year. For iOS draw the four-layer iOS architecture.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Introduction to Mobile Applications

✍️ 2-mark definitions

- **Mobile application (app):** software designed to run on a mobile device such as a smartphone or tablet, usually downloaded from an app store.
- **Native app:** built for one platform using its own language and SDK (Kotlin/Java for Android, Swift/Objective-C for iOS); fastest and has full device access.
- **Web app:** a website designed to work like an app in a mobile browser (HTML, CSS, JavaScript); no installation, limited device access.
- **Hybrid app:** a web app wrapped in a native container (Cordova, Ionic) and distributed through stores.
- **Cross-platform app:** one code base compiled or rendered for several platforms (Flutter, React Native, Kotlin Multiplatform).
- **Progressive Web App (PWA):** a web app that can be installed, works offline with service workers and sends push notifications.
- **Embedded system:** a computer system with a dedicated function inside a larger device, with real-time and resource constraints (washing machine controller, car ECU, smartwatch).
- **App store:** an online marketplace where users find, buy and download apps (Google Play, Apple App Store).
- **Monetization:** the way an app earns money: paid download, freemium, in-app purchase, subscription, advertising, sponsorship.
- **Requirement:** a statement of what the system must do (functional) or how well it must do it (non-functional).
- **Requirement validation:** checking that the documented requirements describe what the users actually need (reviews, prototyping, test case generation).

💡 Core concepts

Native vs web vs hybrid vs cross-platform

Feature	Native	Web	Hybrid	Cross-platform
Language	Kotlin/Java, Swift/Objective-C	HTML, CSS, JS	HTML, CSS, JS in a WebView	Dart (Flutter), JS (React Native)
Performance	Best	Lowest	Medium	Near native
Device access	Full	Limited	Through plugins	Through plugins and bridges
Distribution	App stores	URL	App stores	App stores
Code reuse	None across platforms	Full	High	High
Cost	Highest	Lowest	Low	Medium

Characteristics of mobile devices that shape apps: small screens, touch input, limited battery, variable network (offline, 4G, Wi-Fi), limited CPU and memory compared with desktops, many sensors (GPS, camera, accelerometer), always with the user, personal and private data.

Mobile apps and embedded systems: a smartphone is itself an embedded platform: a SoC (ARM CPU, GPU, modem, DSP), embedded OS (Android/Linux kernel, iOS/XNU), sensors and power management. Many apps talk to other embedded devices: wearables, smart home devices, car infotainment, medical devices through BLE or Wi-Fi.

Market and business drivers for mobile apps

- Huge smartphone user base (India is the second largest smartphone market) and cheap data.
- Mobile-first behaviour: banking (UPI), shopping, food delivery, travel, learning.
- Businesses want direct, personalized engagement (push notifications, loyalty).
- New revenue models: in-app purchases, subscriptions, ads.
- Enterprise mobility: field staff, inventory, CRM on phones.
- New technologies: 5G, AR, AI on device, wearables, IoT.
- Low entry cost: one developer can reach millions through app stores.

Publishing and delivery of mobile apps

Step	Google Play	Apple App Store
Developer account	One-time fee (USD 25)	Annual fee (USD 99)
Build	Signed Android App Bundle (.aab)	Archive signed with distribution certificate and provisioning profile (.ipa)
Testing	Internal, closed, open testing tracks	TestFlight
Listing	Title, description, screenshots, icon, content rating, privacy policy, data safety form	App name, description, screenshots, privacy labels, age rating
Review	Mostly automated, hours to days	Human review against App Review Guidelines
Release	Staged rollout by percentage	Manual or automatic release, phased release
Updates	Increase versionCode	Increase build number

Other delivery: enterprise distribution (Managed Google Play, Apple Business Manager), side-loading APKs on Android, alternative stores.

Requirements gathering

- Stakeholders: users, clients, business owners, developers, testers, regulators.
- Techniques: interviews, questionnaires and surveys, workshops and brainstorming, observation (ethnography), document analysis, competitor app analysis, prototyping, user stories and use cases, focus groups, analytics of existing apps.
- Functional requirements: what the app does (log in, search products, pay).
- Non-functional requirements: performance (launch under 2 s), usability, security, availability, battery use, offline support, accessibility, localization, compatibility with OS versions and screen sizes.

- Mobile-specific requirements: device fragmentation, permissions, connectivity, store policies.

Requirement validation techniques: requirement reviews and walkthroughs, prototyping (paper, wireframes, clickable mock-ups in Figma), model validation, test case generation from requirements, acceptance criteria, checking against characteristics: correct, complete, consistent, unambiguous, verifiable, feasible, traceable.

Examples and comparisons

Example 1: Choosing the app type. A bank wants high security, fingerprint login and the best performance on both Android and iOS: native apps (or Flutter if budget is tight). A college wants a notice board that works on any phone quickly and cheaply: a PWA or responsive web app. A startup wants one code base, store presence and near-native speed: cross-platform (Flutter or React Native).

Example 2: User story with acceptance criteria. "As a student, I want to download notes for offline reading so that I can revise without the Internet." Acceptance criteria: a download button on each note; the note opens in airplane mode; the downloads screen lists stored notes with sizes; the user can delete a download.

Example 3: Store size budget. An app has 18 MB of code and resources and 40 MB of images. Converting images to WebP saves 60% of image size: 40 × 0.4 = 16 MB. New size = 18 + 16 = 34 MB (from 58 MB), which improves install conversion on low-storage phones.

Key points to remember

- App types: native, web, hybrid, cross-platform, PWA.
- Business drivers: users, mobile-first services, engagement, revenue models, enterprise mobility.
- Publishing: account, signed build, testing track, listing, review, release, updates.
- Requirements: functional and non-functional; gathering techniques; validation by review, prototyping, test cases.

⭐ Important questions

1. What is a mobile application? Compare native, web, hybrid and cross-platform apps.
2. Explain the relationship between mobile apps and embedded systems.
3. Explain the market and business drivers for mobile applications.
4. Explain the process of publishing and delivering mobile apps on Google Play and the App Store.
5. Explain requirements gathering and validation for mobile applications.

UNIT 2 Basic Design and Quality Constraints

✍️ 2-mark definitions

- **Embedded OS:** an operating system for an embedded device, small and often real-time, managing limited hardware (FreeRTOS, Zephyr, embedded Linux, Android, iOS, watchOS).
- **RTOS (Real-Time Operating System):** an OS that guarantees tasks finish within deadlines (hard real-time: missing a deadline is a failure; soft real-time: occasional misses are tolerable).
- **Hardware constraint:** a limit imposed by the device: CPU speed, memory, storage, battery, screen size, sensors.
- **Software constraint:** a limit imposed by software: OS version, APIs available, sandboxing, permissions, background execution limits.
- **UI (User Interface):** the screens, controls and feedback through which a user interacts with the app.
- **Touch event:** a low-level input event (finger down, move, up) delivered to the app with coordinates.
- **Gesture:** a meaningful pattern of touch events: tap, double tap, long press, swipe, drag, pinch, rotate, fling.
- **Usability:** how easily and pleasantly users can learn and use the app to reach their goals (effectiveness, efficiency, satisfaction).
- **Availability:** the proportion of time the app and its services are working and reachable.
- **Modifiability:** how easily the app can be changed to fix bugs, add features or adapt to new platforms.

💡 Core concepts

Basics of embedded systems design

- Process: requirements → specification → architecture (hardware and software partitioning) → component design → integration → testing.
- Design metrics: cost (unit and NRE), size, performance, power, flexibility, time to market, correctness, safety.
- Hardware/software co-design: decide which functions run in hardware (speed) and which in software (flexibility).

Embedded OS

OS	Use
FreeRTOS, Zephyr	Microcontrollers, IoT nodes
Embedded Linux	Routers, set-top boxes, gateways
Android	Phones, tablets, TVs, cars (Android Automotive)
iOS, iPadOS, watchOS	Apple devices

OS	Use
QNX, VxWorks	Automotive and industrial real-time

Design constraints for mobile applications

Constraint	Design response
Small screen	Simple layouts, one primary task per screen, responsive design
Touch input	Large touch targets (at least 48 dp Android, 44 pt iOS), gestures
Battery	Batch network calls, avoid wake locks, use JobScheduler/WorkManager, dark theme on OLED
Limited memory	Load images at display size, release resources in lifecycle callbacks, avoid leaks
Unreliable network	Offline cache, retries with backoff, sync later
Fragmentation	Support a range of OS versions and screen densities, test on many devices
Security	Sandboxing, permissions, encrypted storage, HTTPS
Interruptions	Save state on calls and app switches (life cycle)

Architecting mobile applications

- Layers: presentation (UI), business logic (domain), data (repositories, local DB, network).
- Patterns: MVC (Model-View-Controller, classic iOS), MVP (Model-View-Presenter), MVVM (Model-View-ViewModel, Android Jetpack and SwiftUI), Clean Architecture, VIPER (iOS).
- Client types: thin client (logic on server), thick/rich client (logic on device, works offline), hybrid.
- Backend: REST or GraphQL APIs, Backend-as-a-Service (Firebase), push notification services (FCM, APNs).

MVC vs MVP vs MVVM

Pattern	View talks to	Testability	Typical use
MVC	Controller, which updates View and Model	Controller often grows huge ("massive view controller")	UIKit iOS
MVP	Presenter through an interface	Presenter testable without UI	Older Android
MVVM	ViewModel via data binding / observable state	ViewModel testable, survives rotation	Android Jetpack, SwiftUI

User interfaces for mobile apps (design principles)

- Follow platform guidelines: Material Design (Android) and Human Interface Guidelines (iOS).
- Clarity, consistency, feedback, forgiveness (undo), minimal input (auto-fill, pickers), thumb-friendly layout, readable text (at least 16 sp body), contrast and accessibility (screen readers, dynamic type), navigation patterns (bottom navigation, tabs, navigation drawer, back stack).

Touch events and gestures

- Android: `MotionEvent` with actions ACTION_DOWN, ACTION_MOVE, ACTION_UP, ACTION_POINTER_DOWN (multi-touch), ACTION_CANCEL; `onTouchEvent()`; `GestureDetector` (onSingleTapUp, onLongPress, onFling, onScroll) and `ScaleGestureDetector` (pinch).
- iOS: `touchesBegan`, `touchesMoved`, `touchesEnded`, `touchesCancelled` on `UIResponder`; gesture recognizers `UITapGestureRecognizer`, `UIPinchGestureRecognizer`, `UIRotationGestureRecognizer`, `UISwipeGestureRecognizer`, `UIPanGestureRecognizer`, `UILongPressGestureRecognizer`.

Quality constraints

Attribute	Meaning	How to achieve
Performance	Fast launch, smooth 60 fps scrolling, quick responses	Work off the main thread, cache, lazy loading, profile with Android Profiler / Instruments
Usability	Easy to learn and use	Platform guidelines, usability testing, accessibility
Security	Protect data and users	HTTPS/TLS with certificate pinning, encrypted storage (Keystore/Keychain), least permissions, input validation, obfuscation, secure authentication (OAuth, biometrics)
Availability	App and backend reachable	Offline mode, redundant servers, graceful degradation, monitoring, crash reporting
Modifiability	Easy to change	Layered architecture, modules, design patterns, dependency injection, tests, CI/CD

Examples and comparisons

Example 1: Frame budget. At 60 frames per second each frame has 1000 / 60 = 16.7 ms. If list item binding takes 25 ms on the main thread, frames are dropped (jank). Moving image decoding to a background thread brings it under 16.7 ms. At 120 Hz the budget is 8.3 ms.

Example 2: Battery drain. A phone battery has 4000 mAh. An app holds GPS on continuously at 50 mA extra: 4000 / 50 = 80 hours just for GPS; with other use the battery drains much faster. Using fused location with a 1-minute interval and low-power priority at about 5 mA average extends this to 800 hours, so request location only as often as needed.

Example 3: Availability. The backend is 99.5% available. Downtime per month (30 days) = 0.005 × 720 h = 3.6 hours. With an offline cache, users can still read stored data during those hours, so perceived availability is higher.

Example 4: Touch target. On a 420 dpi screen, 48 dp = 48 × 420 / 160 = 126 pixels, which is about 7.6 mm on the screen, comfortably larger than a fingertip contact area.

Code

Detecting a swipe on Android (Kotlin)

```
val detector = GestureDetector(this, object : GestureDetector.SimpleOnGestureListener() {
    override fun onFling(e1: MotionEvent?, e2: MotionEvent,
                         vx: Float, vy: Float): Boolean {
        val dx = e2.x - (e1?.x ?: 0f)
        if (dx > 100) showPrevious() else if (dx < -100) showNext()
        return true
    }
})

override fun onTouchEvent(event: MotionEvent): Boolean =
    detector.onTouchEvent(event) || super.onTouchEvent(event)
```

Pinch gesture on iOS (Swift)

```
let pinch = UIPinchGestureRecognizer(target: self, action: #selector(handlePinch))
imageView.addGestureRecognizer(pinch)

@objc func handlePinch(_ g: UIPinchGestureRecognizer) {
    imageView.transform = imageView.transform.scaledBy(x: g.scale, y: g.scale)
    g.scale = 1
}
```

Key points to remember

- Embedded design metrics: cost, size, performance, power, flexibility, time to market.
- Mobile constraints: screen, touch, battery, memory, network, fragmentation, security, interruptions.
- MVC, MVP, MVVM; three layers: presentation, domain, data.
- Gestures: tap, double tap, long press, swipe, drag, pinch, rotate, fling.
- Quality: performance (16.7 ms frame), usability, security, availability, modifiability.

⭐ Important questions

1. Explain the basics of embedded systems design and embedded operating systems.
2. Explain the hardware and software design constraints for mobile applications.
3. How do you architect a mobile application? Explain MVC, MVP and MVVM.
4. Explain user interface design principles for mobile apps. Explain touch events and gestures.
5. Explain the quality constraints: performance, usability, security, availability and modifiability.

UNIT 3 Advanced Design

✍️ 2-mark definitions

- **Multimedia:** content that combines text, audio, images, animation and video.
- **Codec:** software or hardware that compresses and decompresses media (H.264, H.265/HEVC, VP9, AV1 for video; AAC, MP3, Opus for audio).
- **Streaming:** playing media while it downloads, in small segments; adaptive bitrate streaming (HLS, MPEG-DASH) switches quality to match bandwidth.
- **REST API:** a web interface where an app reads and writes resources with HTTP methods, usually exchanging JSON.
- **GPS (Global Positioning System):** a satellite navigation system that gives the device's latitude, longitude and altitude by measuring signals from at least four satellites.
- **Geofencing:** triggering an action when the device enters or leaves a virtual boundary around a location.
- **OAuth 2.0:** an authorization framework that lets an app access a user's data on another service (Google, Facebook) without seeing their password, using access tokens.
- **Cloud-hosted (cloud-backed) app:** a mobile app whose data, logic or storage lives on cloud services (BaaS such as Firebase, AWS Amplify).
- **Push notification:** a message sent from a server to a device through a platform service (FCM for Android, APNs for iOS) even when the app is not running.
- **Design pattern:** a general, reusable solution to a commonly occurring software design problem.

💡 Core concepts

Multimedia in mobile apps

- Capture: camera and microphone APIs (CameraX on Android, AVFoundation on iOS).
- Playback: ExoPlayer/Media3 (Android), AVPlayer (iOS); support for streaming (HLS, DASH).
- Images: load at the size shown, cache in memory and disk (Glide, Coil, Kingfisher), use WebP/HEIF.
- Design issues: storage space, bandwidth and data costs, battery (hardware decoders), permissions, background playback, accessibility (captions).

Web access from mobile apps

- Options: open the browser, embed a WebView, or call web APIs directly (best for native UX).
- Networking libraries: Retrofit and OkHttp (Android), URLSession and Alamofire (iOS).
- Formats: JSON (most common), XML, Protocol Buffers.
- Rules: never block the main thread (Android throws NetworkOnMainThreadException), use HTTPS, handle errors and timeouts, cache responses, paginate large lists, retry with exponential backoff.

GPS and location

- Sources: GPS (accurate, outdoors, high battery), Wi-Fi and cell tower positioning (less accurate, indoors, low power), fused location provider combines them.

- Permissions: fine vs coarse location; foreground vs background location; user can grant "only while using the app".
- Uses: maps and navigation, ride hailing, delivery tracking, geotagging, geofencing, location-based offers.
- Privacy: ask only when needed, explain why, do not store more precise data than required.

Social networking integration: login with Google, Facebook, Apple (OAuth 2.0 / OpenID Connect), share sheets and share intents, deep links to app content, social graph APIs (with user consent), in-app sharing of scores or posts. Concerns: privacy, platform policy compliance, API changes.

Accessing applications hosted in a cloud computing environment

- App ↔ REST/GraphQL API ↔ cloud backend (compute, database, storage, auth).
- Backend-as-a-Service: Firebase (Auth, Firestore, Cloud Storage, Cloud Functions, FCM, Analytics), AWS Amplify, Supabase.
- Benefits: no server management, scaling, offline sync (Firestore cache), real-time updates.
- Design issues: latency, offline mode, authentication tokens, security rules, cost per read and write.

Design patterns for mobile applications

Pattern	Problem it solves	Mobile example
Singleton	One shared instance	One database or network client for the app
Observer	Notify many when state changes	LiveData/Flow on Android, NotificationCenter and Combine on iOS
Adapter	Make incompatible interfaces work together	RecyclerView.Adapter turns a list of data into views
Builder	Build complex objects step by step	Notification builder, AlertDialog.Builder, Retrofit.Builder
Factory	Create objects without naming exact classes	ViewModelProvider.Factory
Repository	Hide where data comes from	Repository decides cache vs network
Delegate	One object acts for another	UITableViewDelegate, UITextFieldDelegate
Dependency injection	Supply dependencies from outside	Hilt/Dagger on Android
MVC/MVP/MVVM	Separate UI from logic	Architecture of screens

Mobile UI patterns: master-detail, bottom navigation, pull to refresh, infinite scroll, onboarding, empty states, skeleton loading.

Examples and comparisons

Example 1: Adaptive streaming. A user's bandwidth drops from 5 Mbps to 1.2 Mbps. The HLS player switches from the 1080p rendition (4.5 Mbps) to the 480p rendition (1 Mbps) at the next segment boundary, so playback continues without stalling.

Example 2: Data use. A 10-minute video at 2 Mbps uses 2 × 600 / 8 = 150 MB. At 0.8 Mbps it uses 60 MB. Offer a "data saver" setting for mobile networks.

Example 3: Geofence. A shop sets a geofence of radius 200 m. The app registers it with the location service; when the user enters, the OS wakes the app with an enter event and it shows an offer notification, without the app polling GPS.

Code

Calling a REST API with Retrofit (Kotlin)

```
data class Note(val id: Int, val title: String)

interface NotesApi {
    @GET("notes")
    suspend fun list(): List<Note>
}

val api = Retrofit.Builder()
    .baseUrl("https://api.example.com/")
    .addConverterFactory(GsonConverterFactory.create())
    .build()
    .create(NotesApi::class.java)

// inside a ViewModel
viewModelScope.launch {
    val notes = api.list()      // runs off the main thread
    _state.value = notes
}
```

Key points to remember

- Media: CameraX/AVFoundation; ExoPlayer/AVPlayer; HLS/DASH adaptive streaming.
- Web access: never on the main thread; Retrofit/URLSession; JSON; HTTPS; retries.
- Location: GPS, Wi-Fi, cell, fused provider; fine vs coarse; geofencing.
- Social: OAuth 2.0, OpenID Connect, share intents, deep links.
- Cloud: BaaS (Firebase), push (FCM/APNs).
- Patterns: Singleton, Observer, Adapter, Builder, Factory, Repository, Delegate, DI.

⭐ Important questions

1. Explain the design issues of multimedia and web access in mobile applications.
2. How do apps integrate GPS and location services? Explain geofencing.
3. Explain the integration of social networking applications in mobile apps.
4. Explain how mobile apps access applications hosted in the cloud.
5. Explain design patterns used in mobile application development with examples.

UNIT 4 Technology I: Android

✍️ 2-mark definitions

- **Android:** an open-source, Linux-based operating system for mobile devices, developed by Google and the Open Handset Alliance.
- **Android SDK:** the tools, libraries, emulator and APIs needed to build Android apps; used through Android Studio.
- **Activity:** a single screen with a user interface; an app is a collection of activities.
- **Fragment:** a reusable part of the UI with its own life cycle, hosted inside an activity.
- **View:** the basic UI building block that draws on the screen and handles events (TextView, Button, ImageView); **ViewGroup** is a container for views (LinearLayout, ConstraintLayout).
- **Intent:** a message object used to request an action from another component (start an activity or service, deliver a broadcast). **Explicit** names the target class; **implicit** names an action (ACTION_VIEW, ACTION_SEND) and the system finds a component.
- **Service:** a component that performs long-running work in the background without a UI.
- **Broadcast receiver:** a component that responds to system-wide broadcasts (battery low, boot completed).
- **Content provider:** a component that shares app data with other apps through a standard URI interface (Contacts provider).
- **AndroidManifest.xml:** the file that declares the app's components, permissions, minimum SDK and other metadata.
- **APK / AAB:** Android Package (installable file) / Android App Bundle (publishing format from which Play generates optimized APKs).
- **SQLite:** a lightweight, serverless, file-based relational database built into Android and iOS.
- **ART (Android Runtime):** the runtime that executes app bytecode (DEX), using ahead-of-time and just-in-time compilation; it replaced Dalvik in Android 5.

💡 Core concepts

Establishing the development environment: install Android Studio (includes SDK, platform tools, build tools, emulator via AVD Manager), install JDK (bundled), create a project (choose template, language Kotlin or Java, minimum SDK), run on emulator or a USB-debugging-enabled device. Build system: Gradle (`build.gradle.kts`).

Android architecture (software stack, draw it)

Layer (top to bottom)	Contents
System apps	Phone, Contacts, Browser, Camera, your apps
Java API framework	Activity Manager, Window Manager, View System, Content Providers, Package Manager, Notification Manager, Resource Manager, Location Manager, Telephony Manager

Layer (top to bottom)	Contents
Native C/C++ libraries and Android Runtime	Libraries: SQLite, OpenGL ES, WebKit, Media Framework, SSL, libc. ART with core libraries
Hardware Abstraction Layer (HAL)	Standard interfaces for camera, audio, Bluetooth, sensors
Linux kernel	Drivers (display, camera, Wi-Fi, binder IPC), power management, memory management, security

Project structure: `manifests/AndroidManifest.xml`, `java/` (Kotlin or Java code), `res/` (layout XML, drawable, values: strings, colors, themes, mipmap icons), Gradle scripts.

Activity life cycle (most asked)

Callback	When it is called	What to do
onCreate()	Activity is created	Inflate layout, initialize views and data
onStart()	Becoming visible	Start UI-related work
onResume()	In foreground, user can interact	Start camera, animations, sensors
onPause()	Losing focus (dialog, another activity coming)	Pause camera, save light state
onStop()	No longer visible	Release heavy resources, save data
onRestart()	Coming back from stopped	Re-prepare
onDestroy()	Finishing or destroyed by system (rotation)	Final cleanup

Order on launch: onCreate → onStart → onResume. On pressing Home: onPause → onStop. On returning: onRestart → onStart → onResume. On Back: onPause → onStop → onDestroy. Rotation destroys and recreates the activity (save state with `onSaveInstanceState` or a ViewModel).

Views and layouts

Layout	Arranges children
LinearLayout	In one row or column (orientation, weight)
RelativeLayout	Relative to each other or the parent
ConstraintLayout	With constraints, flat and flexible (recommended)
FrameLayout	Stacked on top of each other
TableLayout / GridLayout	In rows and columns
RecyclerView	Efficient scrolling list using a ViewHolder and adapter

Units: dp (density-independent pixels) for sizes, sp (scale-independent pixels) for text. Modern UI can also be written with **Jetpack Compose** (declarative, Kotlin functions marked @Composable).

Interacting with UI: find views (view binding), set listeners (`setOnClickListener`), show feedback (Toast, Snackbar, dialogs), menus (options, context, popup), navigation between screens with intents or the Navigation component.

Persisting data

Method	For
SharedPreferences / DataStore	Small key-value settings
Internal storage files	Private files
External / shared storage (MediaStore)	Photos, documents shared with other apps
SQLite (through Room)	Structured relational data
Network / cloud	Data shared across devices

SQLite on Android: extend `SQLiteOpenHelper` (override `onCreate` to create tables and `onUpgrade` to migrate), use `getWritableDatabase()`, `insert`, `query`, `update`, `delete`, or `rawQuery`. **Room** (Jetpack) adds compile-time-checked SQL with Entity, DAO and Database classes.

Packaging and deployment: set versionCode and versionName, build a release variant, shrink and obfuscate with R8, sign with a release keystore (Play App Signing), produce an AAB, upload to the Play Console, roll out through testing tracks to production.

Interaction with server-side applications: HTTP(S) calls with Retrofit/OkHttp or Volley, JSON parsing (Gson, Moshi, kotlinx.serialization), coroutines or WorkManager for background work, Firebase Cloud Messaging for push, INTERNET permission in the manifest.

Using Google Maps, GPS and Wi-Fi

- Maps: add the Maps SDK, get an API key, add `SupportMapFragment`, implement `OnMapReadyCallback`, add markers, polylines, camera movements.
- GPS: `FusedLocationProviderClient` (`getLastLocation`, `requestLocationUpdates`), with ACCESS_FINE_LOCATION or ACCESS_COARSE_LOCATION runtime permission.
- Wi-Fi: `WifiManager` and `ConnectivityManager` to check network state; ACCESS_WIFI_STATE and ACCESS_NETWORK_STATE permissions; Wi-Fi Direct for peer-to-peer.

Integration with social media: share content through an implicit intent with ACTION_SEND, Google Sign-In / Credential Manager, Facebook SDK login, deep links and App Links to open content in the app.

Android permissions: normal permissions (granted at install, e.g. INTERNET) and dangerous permissions (asked at run time since Android 6, e.g. CAMERA, LOCATION, CONTACTS).

Code

Activity with a button and an explicit intent (Kotlin)

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        findViewById<Button>(R.id.btnOpen).setOnClickListener {
            val i = Intent(this, DetailActivity::class.java)
            i.putExtra("noteId", 42)
            startActivity(i)
        }
    }
}
```

Implicit intents: share text and open a web page

```
val share = Intent(Intent.ACTION_SEND).apply {
    type = "text/plain"
    putExtra(Intent.EXTRA_TEXT, "Revise with Next Engineer")
}
startActivity(Intent.createChooser(share, "Share via"))

startActivity(Intent(Intent.ACTION_VIEW, Uri.parse("https://example.com")))
```

SQLite helper (Java, as often asked)

```
public class DBHelper extends SQLiteOpenHelper {
    public DBHelper(Context c) { super(c, "notes.db", null, 1); }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE notes(id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT)");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldV, int newV) {
        db.execSQL("DROP TABLE IF EXISTS notes");
        onCreate(db);
    }

    public long add(String title) {
        ContentValues v = new ContentValues();
        v.put("title", title);
        return getWritableDatabase().insert("notes", null, v);
    }

    public Cursor all() {
        return getReadableDatabase().rawQuery("SELECT * FROM notes", null);
    }
}
```

Current location (Kotlin)

```
val client = LocationServices.getFusedLocationProviderClient(this)
client.lastLocation.addOnSuccessListener { loc ->
    loc?.let { Log.d("GPS", "${it.latitude}, ${it.longitude}") }
}
```

Manifest entries

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<application android:label="@string/app_name">
    <activity android:name=".MainActivity" android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
    <activity android:name=".DetailActivity" />
</application>
```

Key points to remember

- Stack: Linux kernel → HAL → native libraries + ART → Java API framework → apps.
- Four components: Activity, Service, Broadcast Receiver, Content Provider (all declared in the manifest).
- Life cycle: onCreate, onStart, onResume, onPause, onStop, onRestart, onDestroy.
- Explicit intent names a class; implicit names an action.
- Storage: SharedPreferences/DataStore, files, SQLite/Room, cloud.
- Release: versionCode, R8, signing, AAB, Play Console tracks.
- Maps SDK + API key; FusedLocationProviderClient; runtime permissions for dangerous permissions.

⭐ Important questions

1. Explain the Android architecture with a neat diagram.
2. How do you establish an Android development environment? Explain the project structure.
3. Explain the Activity life cycle with a diagram. What happens on rotation?
4. Explain views, layouts and UI interaction in Android. Explain intents with examples.
5. How is data persisted in Android? Write a program using SQLite.
6. Explain packaging and deployment of an Android app.
7. Explain how Android apps use Google Maps, GPS, Wi-Fi, server-side applications and social media.

UNIT 5 Technology II: iOS

✍️ 2-mark definitions

- **iOS:** Apple's mobile operating system for iPhone, based on Darwin (XNU kernel), with apps built using Xcode in Swift or Objective-C.
- **Objective-C:** an object-oriented superset of C that adds Smalltalk-style messaging (`[object method]`); the original language for iOS and macOS apps.
- **Swift:** Apple's modern, safe, fast programming language (2014) for iOS and other Apple platforms, with type inference, optionals and automatic memory management through ARC.
- **Xcode:** Apple's IDE for building iOS apps, with Interface Builder, simulator, debugger and Instruments.
- **UIKit:** the classic framework for iOS user interfaces (view controllers, views, controls, events); **SwiftUI** is the newer declarative UI framework.
- **View controller (UIViewController):** manages a screen's view hierarchy and its life cycle.
- **Storyboard:** a visual file in Xcode that lays out screens and the segues (transitions) between them.
- **Core Data:** Apple's object graph and persistence framework, usually backed by SQLite, for storing model objects.
- **Core Location:** the framework that gives the device's location, heading and region monitoring.
- **MapKit:** the framework to show maps, annotations and overlays and to get directions.
- **ARC (Automatic Reference Counting):** the compiler inserts retain and release calls so objects are freed when no strong references remain.
- **Provisioning profile:** a file that links an app ID, signing certificate and devices, allowing an app to run on devices or be distributed.

💡 Core concepts

iOS architecture (four layers, draw it)

Layer (top to bottom)	Frameworks
Cocoa Touch	UIKit, SwiftUI, MapKit, EventKitUI, push notifications, gesture recognizers, multitasking
Media	Core Graphics, Core Animation, AVFoundation, Core Image, Metal, Core Audio
Core Services	Foundation, Core Data, Core Location, CloudKit, Contacts, EventKit, SQLite, networking (URLSession)
Core OS	Kernel (XNU), Security framework, Bluetooth, file system, power management, Local Authentication (Face ID/Touch ID)

Introduction to Objective-C

- Files: interface `.h` (declarations) and implementation `.m`.

- Messaging: `[receiver message:argument]`; classes inherit from `NSObject`; `@property` with attributes (strong, weak, nonatomic, copy); `@interface`, `@implementation`; protocols (like interfaces) and categories (add methods to existing classes); blocks (closures).
- Memory: ARC; weak references to avoid retain cycles (delegates are weak).

iOS features: multitasking, push and local notifications, iCloud, Siri and Shortcuts, Face ID/Touch ID, Apple Pay, App Extensions and widgets, strong sandboxing and privacy permissions (Info.plist usage descriptions), Universal apps (iPhone and iPad).

UI implementation: UIKit with storyboards, XIBs or code; Auto Layout constraints for different screen sizes; common views UILabel, UIButton, UITextField, UIImageView, UITableView (lists with data source and delegate), UICollectionView, UINavigationController (push/pop stack), UITabBarController. SwiftUI uses declarative views (Text, Button, List, NavigationStack) and state (`@State`, `@Binding`, `@ObservedObject`).

UIViewController life cycle: loadView → viewDidLoad (one-time setup) → viewWillAppear → viewDidAppear → viewWillDisappear → viewDidDisappear. App life cycle states: not running, inactive, active, background, suspended (UIApplicationDelegate / SceneDelegate callbacks).

Touch frameworks: UIResponder touch methods (`touchesBegan` etc.), UIGestureRecognizer subclasses (tap, pinch, rotation, swipe, pan, long press, screen edge pan), 3D Touch / Haptic Touch context menus, SwiftUI gestures (`onTapGesture`, `DragGesture`, `MagnificationGesture`).

Data persistence using Core Data and SQLite

- UserDefaults (small settings), files in the app sandbox (Documents, Caches), Keychain (secure secrets), Core Data, SQLite directly (C API or wrappers like FMDB, GRDB), SwiftData (newer), CloudKit (iCloud).
- **Core Data stack:** Managed Object Model (.xcdatamodeld with entities and attributes), Persistent Store Coordinator, Managed Object Context (scratchpad where objects are created and saved), Persistent Store (SQLite file). `NSPersistentContainer` sets this up. Fetch with `NSFetchRequest` and predicates.
- Core Data vs SQLite: Core Data manages objects, relationships, undo, change tracking and migration; SQLite is a lower-level relational database with SQL.

Location-aware applications using Core Location and MapKit

- Core Location: `CLLocationManager`, request authorization (`requestWhenInUseAuthorization` with NSLocationWhenInUseUsageDescription in Info.plist), `startUpdatingLocation`, delegate `didUpdateLocations`, region monitoring (geofencing), significant location change service (battery saving), heading.
- MapKit: `MKMapView`, set region, add `MKPointAnnotation`, overlays (MKPolyline), `MKDirections` for routes, `MKLocalSearch` for places.

Integrating calendar and address book with social media applications: EventKit (EKEventStore to read and create calendar events and reminders, with permission), Contacts framework (CNContactStore; replaced the old AddressBook framework), share with `UIActivityViewController`, Sign in with Apple, social SDKs for login and sharing.

Using Wi-Fi: networking with URLSession (data, download and upload tasks, background sessions), Network framework (`NWPathMonitor` to check Wi-Fi/cellular status), Multipeer Connectivity (peer-to-peer over Wi-Fi and Bluetooth), App Transport Security requiring HTTPS.

iPhone marketplace (App Store)

- Join the Apple Developer Program (USD 99 per year).
- Create certificates, App ID and provisioning profiles; set version and build in Xcode.
- Archive the app, upload with Xcode or Transporter to App Store Connect.
- Fill metadata (description, keywords, screenshots, privacy nutrition labels, age rating, pricing).
- Beta test through TestFlight; submit for App Review against the App Review Guidelines.
- Release; monitor ratings, crashes and sales; Apple takes a 15–30% commission on paid apps and in-app purchases.

Swift: introduction and features

- Safe: optionals (`String?`) force handling of nil; strong static typing with type inference; no pointer arithmetic by default.
- Fast: compiled with LLVM.
- Modern syntax: closures, tuples, generics, `guard` and `if let` unwrapping, string interpolation, enums with associated values, pattern matching with `switch` .
- Protocol-oriented programming with protocol extensions; structs and enums are value types.
- Memory managed by ARC; error handling with `do-try-catch` ; concurrency with `async/await` and actors.
- Interoperable with Objective-C in the same project; Playgrounds for experiments; open source.

Objective-C vs Swift

Feature	Objective-C	Swift
Year	1984 (Apple from 1996)	2014
Syntax	Verbose, brackets, header files	Concise, no headers
Safety	Messages to nil silently ignored	Optionals make nil explicit
Type system	Dynamic	Static with inference
Speed	Fast	Generally faster
Memory	ARC (manual before 2011)	ARC
Use today	Legacy code	All new development

Code

Objective-C class

```objc
// Note.h
@interface Note : NSObject
@property (nonatomic, copy) NSString *title;
- (void)show;
@end

// Note.m
@implementation Note
- (void)show {
    NSLog(@"Note: %@", self.title);
}
@end

// usage
Note *n = [[Note alloc] init];
n.title = @"Revise Unit 5";
[n show];
```

Swift basics

```swift
struct Note {
    let id: Int
    var title: String
}

func find(_ id: Int, in notes: [Note]) -> Note? {
    notes.first { $0.id == id }
}

let notes = [Note(id: 1, title: "Android"), Note(id: 2, title: "iOS")]
if let n = find(2, in: notes) {
    print("Found \(n.title)")
} else {
    print("Not found")
}
```

Saving with Core Data (Swift)

```swift
let context = container.viewContext
let note = NoteEntity(context: context)
note.title = "Core Data example"
try? context.save()

let request = NoteEntity.fetchRequest()
let all = (try? context.fetch(request)) ?? []
```

Location with Core Location and a MapKit pin

```
class MapVC: UIViewController, CLLocationManagerDelegate {
    @IBOutlet var map: MKMapView!
    let manager = CLLocationManager()

    override func viewDidLoad() {
        super.viewDidLoad()
        manager.delegate = self
        manager.requestWhenInUseAuthorization()
        manager.startUpdatingLocation()
    }

    func locationManager(_ m: CLLocationManager,
                         didUpdateLocations locs: [CLLocation]) {
        guard let c = locs.last?.coordinate else { return }
        map.setRegion(MKCoordinateRegion(center: c, latitudinalMeters: 500,
                                         longitudinalMeters: 500), animated: true)
        let pin = MKPointAnnotation()
        pin.coordinate = c
        pin.title = "You are here"
        map.addAnnotation(pin)
    }
}
```

SwiftUI screen

```
struct ContentView: View {
    @State private var count = 0
    var body: some View {
        VStack {
            Text("Tapped \(count) times")
            Button("Tap") { count += 1 }
        }
    }
}
```

Key points to remember

- iOS layers: Cocoa Touch, Media, Core Services, Core OS.
- Objective-C: .h/.m, `[obj msg]`, NSObject, @property, protocols, categories.
- View controller life cycle: viewDidLoad, viewWillAppear, viewDidAppear, viewWillDisappear, viewDidDisappear.
- Core Data stack: model, coordinator, context, store (SQLite).
- Core Location (CLLocationManager + Info.plist usage key) and MapKit (MKMapView, annotations, directions).
- EventKit (calendar), Contacts (address book), UIActivityViewController (share).
- App Store: developer program, certificates and profiles, App Store Connect, TestFlight, review.
- Swift: optionals, type inference, value types, protocols, ARC, async/await.

⭐ Important questions

1. Explain the iOS architecture with a diagram.
2. Give an introduction to Objective-C with an example. Compare Objective-C and Swift.
3. Explain UI implementation and touch frameworks in iOS.
4. Explain data persistence on iOS using Core Data and SQLite.
5. How do you build location-aware apps using Core Location and MapKit?
6. Explain integrating calendar, address book and social media, and using Wi-Fi in iOS apps.
7. Explain the process of publishing an app on the iPhone marketplace. Explain the features of Swift.

Last-hour checklist

- ☐ App types table (native, web, hybrid, cross-platform, PWA); mobile device characteristics.
- ☐ Mobile apps and embedded systems; business drivers; publishing table (Play vs App Store).
- ☐ Requirements gathering techniques; functional vs non-functional; validation; user story example.
- ☐ Embedded design metrics; embedded OS; RTOS hard vs soft.
- ☐ Constraints table; MVC vs MVP vs MVVM; UI principles; touch events and gestures.
- ☐ Quality attributes table; 16.7 ms frame budget; battery and availability examples.
- ☐ Multimedia, web access, GPS, geofencing, social integration (OAuth), cloud-hosted apps, push.
- ☐ Design patterns table.
- ☐ Android stack, components, manifest, Activity life cycle table, layouts, intents, storage, SQLite helper, release steps, Maps and GPS, permissions.
- ☐ iOS layers, Objective-C, UIKit and SwiftUI, view controller life cycle, gestures, Core Data stack, Core Location and MapKit, EventKit and Contacts, Wi-Fi, App Store steps, Swift features, Objective-C vs Swift.

Books for this subject

The AKTU syllabus copy we could access did not show the book list for this elective, so these are the standard textbooks that cover its units. Check your college's copy of the syllabus for the official list. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Professional Mobile Application Development	Jeff McWherter, Scott Gowell	Wrox / Wiley	WorldCat
2	Android in Practice	Charlie Collins, Michael Galpin, Matthias Kaeppler	Manning	WorldCat
3	Beginning Objective-C	James Dovey, Ash Furrow	Apress	WorldCat

#	Title	Author(s)	Publisher	Where to get it legally
4	Beginning iOS Development: Exploring the iOS SDK	David Mark, Jack Nutting, Jeff LaMarche, Fredrik Olsson	Apress	WorldCat
5	Embedded Systems: Architecture, Programming and Design	Raj Kamal	McGraw Hill Education	WorldCat

The official Android developer guides (developer.android.com) and Apple's Swift book and developer documentation (docs.swift.org, developer.apple.com) are free and cover Units 4 and 5 in full.

Syllabus source: AKTU B.Tech CS/CE and CSE 4th year syllabus, 2025-26 (copy hosted by an affiliated college, PDF), with the elective unit list cross-checked against aktunotes.in.

NEXT ENGINEER
sheerat